LM 기반 영어 명령 해석과 자동차 제어

프로젝트 계획서 · 2026.10.07 · 발표 2026.10.20

영어 단어·짧은 문장을 메인 Arduino Uno R3가 받아 두 계산 Uno에 보낸다. 두 보드의 신경망 계산 결과를 합친 뒤, 메인 D2/D3 software UART로 제어 Uno에 명령을 전달한다. 제어 Uno는 전진·후진, 조명, 와이퍼 서보, 피에조 경적을 담당한다.

이번 계획의 목표

LM은 프로젝트의 언어 모델 주제를 뜻한다. 실제 초기 알고리즘은 짧은 영어 문장을 10개 의도로 분류하는 작은 학습형 신경망이다. 자유 문장을 생성하는 LLM·ChatGPT·SmolLM을 보드에 올리는 계획은 아니다. 한국어는 지원하지 않는다.

구분	계획	현재 상태
언어 해석	10의도 단어 모델 학습·2보드 export	V3 학습·export·Uno probe·독립감사 완료
자동차 제어	모터 1개 시험대 → 차체, LED, 와이퍼 서보, 피에조 경적	피에조 종류·부품 미정·실물 미검증
통신	메인에서 UART·I2C·SPI 실제 전송 중첩	배선·파형 검증 예정

과제 원문

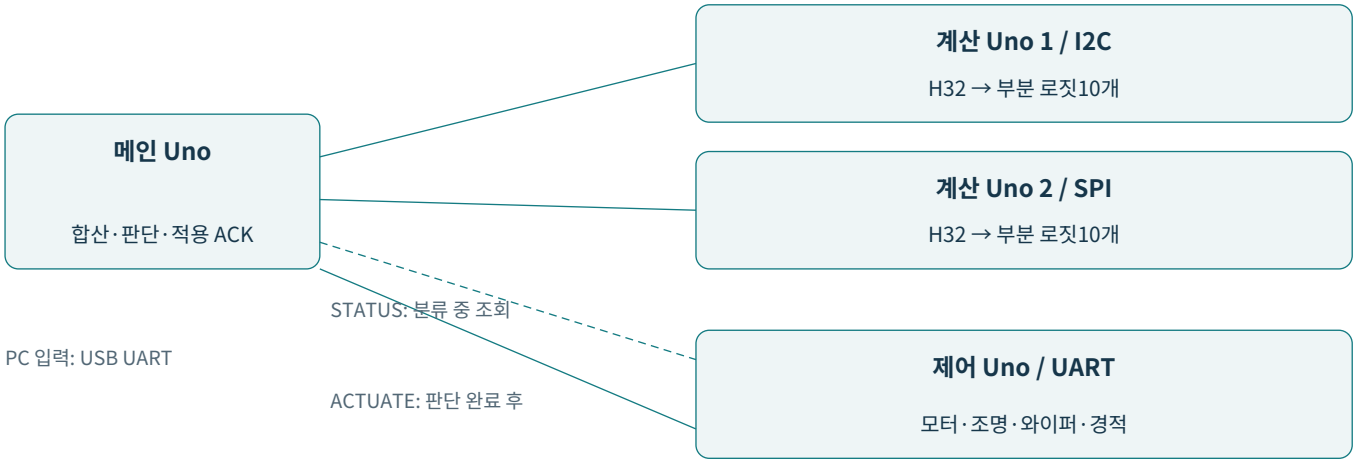
“Using 4 arduinos, create a cooperating application that utilize all 3 communication methods. (UART, SPI, I2C) At least 1 arduino uno should run all three protocols at the same time. The reports can be the same for the teams. However, individual reports should contribution (yes/no) for each of the members (excluding yourself) on the first page. Prepare for presentation on Oct 20. The presentation must be under 5 minutes.”

성공의 정의

네 보드가 영어 명령의 해석과 실제 구동을 협력 수행한다. 메인은 I2C·SPI 추론 전송 중 UART로 제어 보드의 현재 출력상태를 조회하여 세 전송의 공통 중첩 구간을 기록한다. 새 구동 명령은 분류 결과가 확정된 뒤에만 보낸다. 출력은 일치하는 적용 ACK를 받은 뒤 APPLIED로 표시한다.

호스트 모델 학습은 완료되었고 실물4보드 자동차 제어는 구현·검증 계획이다. 이름·팀 정보는 포함하지 않는다. 최종 개인 보고서 첫 페이지에는 본인을 제외한 각 팀원의 기여 여부 Yes/No를 작성해야 한다.

시스템 흐름: 두 계산 보드와 한 제어 보드



실선: 계산·최종 제어 | 점선: 동시 상태 조회 | 부품·실물 검증 예정

입력부터 물리 동작까지

영어 입력 → 길이·어휘 검사와 정규화 → I2C/SPI 부분 로짓 → 메인 합산·의도 판단 → UART ACTUATE → 제어 Uno의 상태 적용 → ACK → 사용자에게 APPLIED 표시 순서이다. 신경망의 자유 텍스트를 GPIO 신호로 해석하지 않고, 허용된 의도 enum만 전달한다.

점선 STATUS 경로는 분류 중 상태 조회용이다. 실선 ACTUATE 경로는 두 계산 보드의 결과를 받은 이후의 동작 명령이다. 계산 중에 전진·후진 결과를 추측하여 먼저 구동하지 않는다.

구성	보드·모델 배치	과제 충족
계산 Uno 1대	메인 + H64 전체 계산 + 제어 = 3대	개발·비교 기준. 4대·3프로토콜 조건 미충족
계산 Uno 2대	메인 + I2C H32 + SPI H32 + UART 제어 = 4대	최종 목표. 동일 H64 모델 분할

역할의 경계

PC는 학습·export·업로드·검증 및 입력/표시에 사용한다. 실행 중 영어 의도의 추론 산술과 제어는 Uno에서 수행한다. 두 계산 보드가 세 번째 UART 계산 보드를 요구하는 구조가 아니다. UART는 자동차 제어 보드와 연결한다.

메인은 USB 하드웨어 UART D0/D1로 입력을 받고, 별도 software UART D2 RX/D3 TX로 제어 보드와 통신한다. 계산 보드의 I2C와 SPI는 각자의 부분 점수를 반환한다. 제어 보드는 GPIO/PWM·서보 setpoint·소프트웨어 상태를 ACK/STATUS로 보고한다. APPLIED는 출력 적용을 뜻하며 센서로 물리 동작을 확인했다는 뜻은 아니다. 실제 이동·회전은 영상·관찰로 따로 검증하고 센서는 향후 선택 사항이다.

영어 의도 데이터와 학습형 모델

같은 의도, 다른 영어 표현

최대96 ASCII 바이트·16토큰의 한 줄을 받으며 초과 입력은 자르지 않고 거절한다. 소문자·공백 압축·끝의 .!? 제거 후 학습 어휘의 단어 ID로 변환한다. 지원 밖 문자·축약형·OOV는 거절한다. 의미가 있는 not를 삭제하지 않는다.

ID / 의도	학습 데이터에 포함한 유사 표현 예시
0 FORWARD	go forward / move ahead / drive forward / please move forward / can you move ahead
1 REVERSE	move backward / go backwards / reverse / back up / please move back
2 STOP	stop / stop moving / halt / stop the car
3 LIGHT_ON	switch on the lights / turn the headlights on / lights on
4 LIGHT_OFF	switch off the lights / turn the headlights off / lights off
5 WIPER_ON	start the wiper / turn on the wipers / wipe the windshield / start wiping
6 WIPER_OFF	stop the wiper / turn off the wipers / stop wiping
7 UNKNOWN	do not roll forward / if it rains move forward / never silence the buzzer
8 HORN_ON	turn on the horn / sound the horn / start honking / honk
9 HORN_OFF	turn off the horn / stop honking / silence the horn / stop the horn

문장군을 나눈 학습·평가

합성41,121문장을 train31,202 / validation4,251 / frozen test5,668로 나누었다. 영어 단어·공손한 접두어·동의어의 조합을 만들고, 같은 template family는 같은 split에만 넣었다. 데이터는 사람이 작성한 규칙의 합성 변형이며 실제 사용자의 자연어 corpus는 아니다. 알려진 단어의 새 조합을 시험하고, 처음 보는 단어는 OOV로 거절한다.

학습 어휘247개, PC에서 학습한 word embedding16 → mean pool → H64 ReLU → 10 logits 모델을 양자화·export한다. mean pool은 단어 순서를 표현하지 못한다. 부정·복합·조건부 명령은 입력 guard와 UNKNOWN 데이터로 제한하며, 자유로운 영어 이해나 일반 LLM의 성능을 주장하지 않는다. seed20261007·1,600단계에서 validation loss로 checkpoint를 선택했다. frozen test는 학습·checkpoint·임계값 선택에 사용하지 않았다. 동작 핵심 구절·알려진 어휘는 split 간 공유한다.

판단·거절·빠른 정지

confidence0.90·margin0.20은 validation 데이터로만 선택했다. softmax 값이 안전성이나 실제 확률을 보장하지 않는다. UNKNOWN·낮은 확신·부정·복합 명령은 ALL_STOP으로 모터·조명·와이퍼·경적을 모두 끈다. “go forward and turn lights on”은 초기 한 입력 한 동작 원칙에서 거절한다.

향후4보드 메인에는 정확한 stop의 빠른 ALL_STOP 경로를 둔다. 현재 의도 probe는 exact-stop shortcut 없이 모델로 분류한다. 라벨 버전2는 기존 ID0~7을 유지하고 경적 ON/OFF를8/9로 추가한다. not/and·조건부 형태는 보수적 입력 guard로 거절하고 그 역할을 모델 성능과 구분해 공개한다. 의도 분류 자체는 학습형 모델로 수행하며, 부정문을 전진으로 바꾸지 않는다.

신경망 분할과 export: 학습된 단어 모델

전체 모델과 텐서

최대16토큰의 embedding16 평균 → H64 ReLU → 10의도 로짓. 단어 순서가 평균에서 사라지므로 한 동작 표현의 조합에 범위를 제한한다.

배열	전체 모델	I2C / SPI 분할
e	[247,16], learned vocabulary	각 보드에 전체 복제
w / b	[16,64] / [64]	w[:,0:32], b[0:32] / w[:,32:64], b[32:64]
o	[64,10]	o[0:32,:] / o[32:64,:]
c / scale	c[10], se/sw/so	c는 메인에만, scale은 shard에서 유지

각 보드는 pk = ReLU(mean(E[token IDs]) · Wk + bk) · Ok의10개 float (40 B)를 반환한다. 메인은 c + pI2C + pSPI로 bias를 한 번만 더한다.

저장 데이터 예산 - 실제 빌드와 구분

int8 e/w/o와 float32 b/c를 사용한다. 전체 저장 배열+scale은5,924 B (e3952+w1024+o640+b256+c40+scale12). 각 H32 shard는4,924 B (e3952+w512+o320+b128+scale12), 두 shard+메인 bias40은9,888 B이다. 메인 token lexicon · 메타데이터 · 통신 코드/버퍼 · 스택은 별도이며 이 값은 Flash/SRAM 실측이 아니다.

합성 평가와 독립 감사의 구분

V3 평가 / 범위	측정 결과
합성 test5,668 raw int8 macro-F1	1.00000
합성 guard F1 / 정상명령 수용률	0.99866 / 99.8409%
비중복 감사 raw NN (어휘 내42/49)	39/42 정확 (92.86%), UNKNOWN3개 오분류
전처리+신경망+거절정책 (비중복49)	정상34/34 · UNKNOWN15/15 정확, 위험 수용0

새 감사51개에서 TRAIN 중복2개를 제외했다. 어휘 내42개 raw NN의39개가 정확했고 UNKNOWN3개 오분류는 일반 guard로 거절했다.7개는 전처리 거절이어서 전체 pipeline은49개 모두 정확했다. 순수 신경망100%라는 뜻이 아니며 작은 범위의 감사이다. 이전 감사들은 회귀시험이며 모든 영어 · 새 동의어 · 차체 성능은 보장하지 않는다.

export와 동일 모델 확인

intent/uno_probe/src/generated/에 양자화 모델 · H32 shard · main bias를 저장한다. V3 model ID30918170 · 어휘 hash · 라벨 버전2 · 차원 · 담당 범위는 intent/metadata.json과 함께 고정한다. 원래 scale을 유지해 shard별 재양자화는 하지 않는다. 메인은 hidden0~63이 중복 · 누락 없이 덮이는지 검사한다.

로컬 근거: intent/metadata.json · metrics.json · training_log.json · intent/INDEPENDENT_VERIFICATION.md (감사 · AVR/C++), intent/verification/ (재현 도구)

배선과 전원: 보드별 핀·타이머 분리

메인과 통신 핀 [1]

링크	메인 Uno	상대 Uno	계획 설정
USB 입력	D0 RX / D1 TX	PC 시리얼 모니터	9600 baud
제어 UART	D2 RX / D3 TX	제어 D1 TX / D0 RX, 교차 연결	9600, 8N1
계산 I2C	A4 SDA / A5 SCL	같은 핀, 주소 0x12	100 kHz·pull-up
계산 SPI	D10 SS, D11 MOSI, D12 MISO, D13 SCK	같은 핀, slave D10 INPUT	Mode 0, MSB first, 125 kHz

메인 D10은 OUTPUT이다. 제어 보드는 UART 업로드 시 D0/D1 외부 연결을 분리하고, 정상 동작 중 USB 시리얼 모니터를 열지 않는다. 메인의 Timer1은 software UART용으로 예약한다. SPI D13은 SCK이므로 활동 LED로 쓰지 않는다.

제어 Uno의 출력 핀 - 부품 선정 후 확정

기능	제어 Uno 핀	연결·제약
모터 1개 H-bridge	D4 IN1, D7 IN2, D5 PWM enable	모터를 GPIO에 직접 연결하지 않음
조명	D8	LED는 220~330Ω 저항, 큰 조명은 별도 드라이버
와이퍼 서보	D9 signal	Servo는 Uno Timer1 사용; D9/D10 analogWrite 제한 [6]
피에조 경적	D6 signal	수동 피에조 tone/noTone, 부품별 드라이버 확인 [9]
선택적 로컬 정지 버튼	D2 INPUT_PULLUP	모든 출력 OFF, 메인 D2와는 다른 보드

Servo 타이머는 제어 보드에 있고 software UART 타이머는 메인에 있다. 제어 UART는 하드웨어 UART를 사용한다. 모터 PWM D5는 Timer0로 유지한다. Uno tone은 Timer2를 써 D3/D11 PWM에 영향을 줄 수 있다 [9]. 제어 보드의 Servo Timer1·모터 D5에는 겹치지 않는다. 두 모터 차체는 추가 핀·드라이버 예산을 다시 잡고, 먼저 모터 1개 시험대로 전진/후진을 확인한다.

전원과 부품의 결정 사항 [7, 8]

모터 전압·stall current에 맞는 H-bridge를 선정한다. TB6612/L298N은 검토 예시이며 현재 선택된 부품은 없다. TB6612를 선택하면 STBY용 D12 또는 부품별 enable 조건을 추가로 확정한다. 보호 회로와 방향 truth-table은 선택한 드라이버 데이터시트로 확인한다.

모터·서보는 적절한 외부 전원/레귤레이터와 드라이버로 공급하고 Uno와 공통 GND를 연결한다. 모터를 Uno GPIO/5 V에 직접 공급하지 않는다. 독립 USB 전원의 5 V 레일을 서로 묶지 않는다. 서보 각도·구동 전압·기구 간섭은 실물 부품 결정 후 확인한다. 피에조는 수동형을 잠정 가정해 tone(D6,1000)/noTone(D6)를 사용한다. 능동 부저면 디지털 ON/OFF로 바꾸며 전압·전류·드라이버 필요 여부는 부품 선정 후 확인한다.

공통 32바이트 패킷과 작업 상태

프레임 규격 - 구현 예정

바이트	필드	규칙
0 / 1 / 2	SOF / version / op	0xA5 / 1 / 허용된 명령 코드
3~4 / 5 / 6	id / seq / payloadLen	uint16 little-endian / 청크 번호 / 유효 길이
7~30	payload[24]	뒤쪽 0 padding
31	CRC8	poly 0x07, init/xorout 0, non-reflected; 앞31 B 검사

계산 op는 LOAD_INPUT, START, STATUS, GET_RESULT, RESULT, ACK, ERROR이다. 메인은 영어를 token ID로 전처리해 두 계산 보드에 동일하게 보낸다. 입력은16개 uint16 LE token ID, 총32 B로24+8 B 분할한다. PAD=0xFFFF는 평균에서 제외하고 모든 나머지 ID는0~246인지 검사한다. 결과40 B는24+16 B로 나눈다. float는 sizeof(float)==4·IEEE754 호환성을 확인하고 memcpy로 little-endian binary 직렬화한다. UART 제어 op는 ACTUATE, ACK, STATUS, HEARTBEAT, STOP이며 같은 envelope를 사용한다.

ID와 중복 방지

계산 job ID와 구동 command ID는 op별 의미를 구분한다. 계산 job ID 재사용 전에는 진행 중 작업을 종료하고 링크를 drain한 뒤 두 계산 보드를 reset/handshake하여 옛 결과가 남지 않게 한다. 시작 STATUS로 모델·어휘·라벨 ID·차원·shard 범위를 확인한다. 보드별 결과2-bit chunk mask로 중복 RESULT를 무시하고 2청크 완료 후 한 번만 합산한다.

UART 제어 payload는 session/epoch uint32 4 B + intent1 B + duration uint16 2 B =7 B를 기본으로 하며 필요한 출력상태·버전도24 B 안에 넣는다. 제어 보드는 모든 ACTUATE/STOP/HEARTBEAT/STATUS에서 현재 session과 command ID를 확인한다. ACK는 session·command ID·적용출력상태를 되돌린다. 새 세션은 정지 상태의 handshake로 시작하고 캐시·pending 명령을 모두 무효화하며 재개하지 않는다. 같은 ID는 캐시 ACK만 반환하여 구동·duration을 재시작하지 않는다. 잘못된 op·intent·길이·CRC는 실행하지 않는다. session prefix는 UART 제어에만 적용하며 계산 token 입력32 B는24+8 분할한다.

링크별 송수신

링크	처리 방식
I2C	주소 0x12, Wire 최대32 B. onReceive는 버퍼·seq만 갱신, onRequest는 캐시된 frame만 반환 [2].
SPI	SS 아래 요청 → 응답 준비 확인 → 별도 read에서 dummy bytes. SPDR preload·첫 byte pipeline·slave ISR용 간격을 검증한다.
UART	메인 timer UART·제어 hardware UART, 유효 용량64 B 이상 ring buffer. SOF 재탐색·CRC·ACK 흐름 제어.

계산 FSM은 IDLE → RECEIVE → COMPUTE → RESULT_READY이다. 신경망은 main loop에서 계산하고 ISR/콜백은 짧은 버퍼 입출력만 수행한다. 통신 timeout과 분류 작업 deadline 2~10초 후보는 별도로 교정한다. 같은 ID의 재시도 최대2회 이후 명시적 실패로 종료한다. 실패·UNKNOWN은 ALL_STOP이며 기존 동작을 자동 재개하지 않는다.

메인: 계산 결과 이후에만 동작 명령

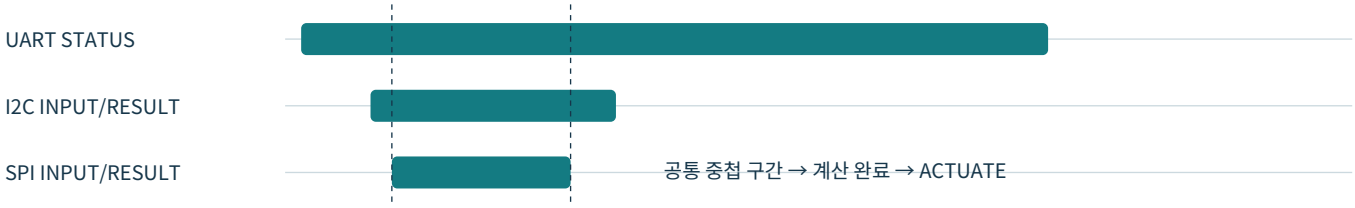
분류·제어의 의사코드 - 구현 예정

```
input = read_line(maximum 96 ASCII bytes, 16 tokens)
validate_and_normalize(input)
if input == "stop": enqueue_priority_STOP(); wait_matching_ACK()
else:
    begin job(modelId, immutable tokens, new jobId, stopGeneration)
    totals = output_bias; receivedMask = 0
    dispatch LOAD_INPUT + START to I2C and SPI
    while receivedMask != 0b11:
        poll_I2C(); poll_SPI(); poll_UART_status(same jobId)
        if complete matching partial10 and not already counted:
            totals += partial10; set worker bit
        if timeout/model mismatch: send_STOP(); fail job
    intent = argmax(totals); evaluate confidence/margin
    if UNKNOWN or rejected: intent = STOP
    if STOP generation changed: abort job; do not actuate
    send_UART_ACTUATE(intent, new commandId, duration)
    if matching ACK reports applied state: print APPLIED
    else: print ERROR; do not claim actuation succeeded
```

같은 job/model ID의 두 계산 결과를 모두 받아야 한다. tokens[16]은 작업 중 불변이며 totals[10] float와 청크 상태를 고정 버퍼로 둔다. 한 보드의 최상위 결과만 채택하지 않는다. 받은10개 float가 모두 finite인지 검사하고 NaN/Inf면 ERROR+STOP으로 처리한다. 분류 완료 전에 새 모터 동작을 보내지 않는다. 입력 거절도 ALL_STOP으로 처리한다. STOP은 pending 분류 job을 취소하고 stopGeneration barrier를 올려 늦은 결과가 새 ACTUATE를 만들지 못하게 한다. 제어 보드는 STOP 이전 command ID와 대기 park/deadtime을 무효화한다.

세 실제 전송의 공통 구간 확보

계획 타임라인 · 실측 아님 · 같은 분류 job의 실제 상태/모델 전송



STATUS는 구동하지 않음 · 새 ACTUATE는 두 점수 수신 이후

추론의 입력/결과 거래 중 UART STATUS 또는 HEARTBEAT으로 제어 Uno의 현재 출력상태를 확인한다. 이 거래에는 같은 분류 job tag를 넣고, I2C/SPI 실제 모델 전송과 세 가지가 겹치는 하나의 공통 시간 구간을 기록한다. STATUS/HEARTBEAT은 모터·조명·와이퍼·경적을 바꾸거나 drive lease를 연장하지 않는다.

최종 ACTUATE는 계산 완료 뒤의 별도 UART 거래이다. 동시성 증거를 만들기 위해 아직 분류되지 않은 방향을 보내지 않는다. 단순한 별도 ping이나 계산만 겹친 로그는 과제 통과 증거가 아니다.

비차단 엔진과 제어 보드의 동작

메인 통신 엔진 - 구현 예정

엔진	초기 설정·검증
software UART	Timer1 /8, OCR1A=207: 약104 µs bit·9615 baud, 약0.16% 오차. D3 PORTD TX, D2 INT0 시작 검출·compare B 중앙 RX sampling.
I2C / TWI	16 MHz, TWBR=72, TWPS=0: 100 kHz. ISR/state machine으로 start/address/data/stop 진행 [5].
SPI	SPE·MSTR, SPR1:SPR0=11, SPI2X=0: /128 =125 kHz. SPIF poll·바이트 간격, Mode0·MSB first [4].

전체 프레임 동안 cli()를 유지하는 기본 SoftwareSerial TX는 사용하지 않는다 [3]. blocking Wire.endTransmission/전체 SPI 전송 대신 엔진별 짧은 단계로 loop에 복귀한다. Timer1 충돌·RX 위상·ISR 지터·유실 바이트를 파형으로 검증한다. 레지스터 설정만으로 동시성 달성을 주장하지 않는다.

제어 Uno: 독립적인 비차단 FSM

기능	동작 계획
전진 / 후진	낮은 PWM·유한 duration. 반대 방향 요청 시 PWM0 → 선택한 driver의 neutral → 약100 ms deadtime 후보 → 새 방향.
조명	ON/OFF를 독립 latch. 다른 정상 명령이 조명을 초기화하지 않음.
와이퍼	millis 기반 20~30 ms마다 2~5° 이동, 45~135° 초기 후보. 기구 간섭 확인 후 조정.
WIPER_OFF	왕복을 중지. 필요 시45°로 천천히 park하는 옵션을 설정.
경적	HORN_ON은 tone을 latch, HORN_OFF는 noTone. 부품 미선정인1000 Hz는 후보.
STOP / 로컬 ESTOP	모터 PWM0·조명 OFF·와이퍼 OFF·noTone. STOP 후 park 동작도 금지.

부팅 시 모터 정지·조명 OFF·와이퍼 OFF·경적 OFF이다. STOP은 방향 전환·와이퍼 park·대기 actuation을 취소하고 이전 job/session의 늦은 명령으로 출력을 다시 켜지 않는다. delay 왕복 루프를 사용하지 않고 UART 수신과 STOP 처리를 계속한다. 정확한 stop 입력은 현재 UART 프레임을 마친 다음 우선 전송하며 프레임 중간을 끊지 않는다. 로컬 정지 버튼은 메인·분류 실패와 독립적으로 동작한다.

구동 시간의 제한과 오류 후 상태

초기 데모 DRIVE는 500 ms duration, 최대1000 ms cap 후보로 자동 정지한다. 실제 속도·시간은 시험대에서 교정한다. 제어 보드의 자체 타이머가 메인 정지·분류 지연과 무관하게 만료를 처리한다. 같은 명령 ID의 재전송이나 500 ms HEARTBEAT은 duration을 갱신하지 않는다.

연속 구동을 추가한다면 현재 명시적으로 승인된 drive intent에 한해서만 새 command ID로 갱신한다. UNKNOWN·timeout·새 분류 실패 뒤에는 자동 갱신하지 않는다. 잘못된 패킷은 실행하지 않으며, 오류 ALL_STOP 정책과 DRIVE 만료 시 모터 정지를 시험한다.

구현 단계와 예정 파일 구조

모듈의 책임 - 호스트 학습과 미래 펌웨어 구분

예정 경로	책임
firmware/main/main.ino	입력 정규화·합산·거절·ACK·비차단3링크
firmware/compute_i2c/compute_i2c.ino	I2C frame·H32 부분 추론·결과 캐시
firmware/compute_spi/compute_spi.ino	SPI pipeline·H32 부분 추론·결과 캐시
firmware/actuator_uart/actuator_uart.ino	hardware UART·모터/LED/Servo/피에조 FSM·만료
firmware/shared/protocol.h / neural_shard.h	ID·CRC·enum / PROGMEM 부분 계산
intent/train.py / reference.py	10라벨 단어 모델 학습·전처리·호스트 추론
intent/dataset/ · intent/uno_probe/src/generated/ · intent/tools/	그룹 split 데이터·모델/shard·검증 도구

단계별 통과 조건

단계	작업	저장할 증거
A 데이터·모델	V3 학습·probe·독립감사 완료	41,121문장·split·metadata·오분류표
B 호스트 분할	H32+H32 로짓 합·선택 의도 비교	50입력 CSV·오차·라벨 ID
C 세 링크	32 B ACK/CRC 왕복·STATUS 현재 출력상태	UART/I2C/SPI 로그·중첩 trace
D 제어 시험대	모터1개·LED·서보·피에조 개별 → ACK 연결	핀·driver 설정·동작 영상
E 통합·오류	영어→의도→UART→실물·누락/CRC/만료	APPLIED 로그·복구·정지 확인
F 비교·발표	1계산/2계산 동일모델 지연·메모리	p50/p95·4빌드·스택·발표 영상

기존 tools/test.sh와 tools/build.sh는 문장 생성 기준 코드를 검증한다. 새 자동차 제어 파일이 구현되면 각 스케치에 arduino-cli compile --fqbn arduino:avr:uno를 적용한다. 새 intent/ 호스트 모델 도구와 미래4보드 firmware/를 구분한다. 실제 실행한 학습·검증의 설정과 결과를 metadata/report에 기록한다.

부품이 정해지면 드라이버 truth-table·PWM·전원·서보 기구 한계를 먼저 확정한다. 두 모터 차체 확장은 모터1개 시험대·통신·정지 시험이 통과한 이후에 진행한다.

PROGMEM 가중치는 pgm_read_byte/pgm_read_float로 읽는다. 향후4보드의 입력→판단, 판단→ACK, 입력→APPLIED p50/p95·전체 Flash/SRAM·스택 high-water는 별도 측정한다. 기존 문자 생성 Flash8614 B/SRAM462 B는 새 모델 측정값이 아니다.

시험 계획: 의도·통신·동작을 각각 확인

정확도와 수치 동등성

시험	예정 통과 기준·관찰 항목
학습 품질	문장군 분리 test set, macro-F1·라벨별 recall·confusion matrix·UNKNOWN 거절률
1계산 vs2계산	고정50입력, 로짓 abs/rel 1e-4 이내·선택 의도 일치
영어 경계	대문자·공백·96/97 B·16/17토큰·비ASCII·지원 밖 문자·긴 줄 복구
부정·복합	go not forward, 두 동작 요청을 동작 거절로 분류
교정	confidence/margin 교정, test 위험한 동작0건 목표(측정 결과 아님)

오류·구동·동시성

시험	확인할 실제 행동
CRC / 미허용 enum / 옛 session	동작하지 않음·ERROR 또는 정책상 STOP
중복 ACTUATE	캐시 ACK, 모터 duration 재시작 없음
계산 보드 누락 / 모델 불일치	한 보드 fallback 없음·ALL_STOP·다음 입력 복구
분류 지연 / 메인 종료	제어 Uno의 DRIVE 만료·자동 정지, 재개 없음
정지·방향 전환	STOP·로컬 버튼으로4기능 OFF, park/늦은 명령 재개 없음, UART 수신 유지
ACK / 물리 동작	APPLIED는 출력 적용 ACK, 실제 이동·서보 왕복은 영상·관찰로 별도 검증
3프로토콜 동시성	UART STATUS + 실제 I2C/SPI 추론 전송 공통 구간

로직 분석기는 UART TX, I2C SDA/SCL, SPI SCK/MOSI/SS 6채널을 기록한다. 필요 시 UART RX·SPI MISO를 추가하여8채널로 확장한다. job tag와 디코딩된 frame을 trace CSV/VCD·스크린샷으로 저장한다. heartbeat와 STATUS는 제어 보드의 출력상태를 확인하는 실제 거래이며 새 actuation을 실행하지 않는다.

실제 Uno probe 빌드와 미래4보드 비교

의도 출력 probe	Flash (32,256 B 중)	전역SRAM (2,048 B 중)
전체 모델	14,340 B	413 B
I2C / SPI 각 H32 shard	12,620 B	425 B

이 빌드는 token 사전·guard·추론·출력용이며 제어 핀과 실제 I2C/SPI/UART4보드 동작을 구현하지 않는다. 독립 C++471입력의 결정 불일치0건·점수 최대오차7.62e-6, 실제 export shard431입력의 합산 최대오차1.48e-5를 확인했다. 런타임 스택은 실물 미측정이다.

10월 20일 발표까지의 실행 계획

단계별 일정과 산출물

기간	작업	통과 산출물
10/7~9	영어 데이터·H64 모델·부품/시험대 결정	label·split·model ID·핀/전원 계획
10/10~12	I2C/SPI 부분 추론·UART 제어·ACK	50입력 비교·각 링크·개별 구동
10/13~15	4보드 통합·STATUS 동시통신·정지/만료	중첩 trace·영어→ACK→동작
10/16~18	오류·부정/복합·지연·메모리 시험	시험표·오분류·빌드·스택·영상
10/19	4분40초 리허설·자료·최종 보고서	타이머·영상·타인Yes/No 기여 표시
10/20	5분 미만 발표	실물 동작·추론/동시통신 증거·제한

발표 스토리보드: 총4분40초

시간	내용	화면·증거
0:00~0:35	영어 명령으로 자동차 기능 제어	문제·과제4대/3통신 요건
0:35~1:15	10의도 모델·1계산/2계산	데이터·분할·거절 정책
1:15~2:10	4보드·핀·결과 이후의 구동	구조도·ACTUATE/STATUS 구분
2:10~3:20	전진/후진·조명·와이퍼·경적·STOP 시연	영어입력·적용출력 ACK·동시trace
3:20~4:05	동등성·오류·시간·메모리	실측 비교·만료/CRC·오분류
4:05~4:40	결론·제한·기여	달성 요건·부품/모델 범위·역할

현장 시연과 제출 자료

한 입력 한 동작으로 짧게 시연하고, 전진·후진은 교정된 낮은 속도와 유한 duration으로 시험한다. 영상은 현장 실패에 대비한 기록이며, 실제4보드 동작과 세 실제 전송 중첩을 확인하는 요건을 대체하지 않는다. 미달성 항목은 완료로 적지 않는다.

최종 개인 보고서 첫 페이지에는 본인을 제외한 각 팀원의 Yes/No 기여 표시를 추가한다. 발표·보고서에 실제 사용한 모델, 데이터, 펌웨어와 부품 정보를 기록한다. 새로운 제어 모델과 기존 문장 생성 데모를 구분한다.

참고 자료와 현재 증거의 경계

공식 기술 자료

[1] Arduino Uno R3 데이터시트: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf

[2] ArduinoCore-avr Wire.h (32 B 버퍼): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/Wire.h

[3] ArduinoCore-avr SoftwareSerial.cpp (TX/cli): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SoftwareSerial/src/SoftwareSerial.cpp

[4] ArduinoCore-avr SPI.h (clock divider): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SPI/src/SPI.h

[5] ArduinoCore-avr twi.c (TWBR 설정): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/utility/twi.c

[6] Arduino Servo AVR 타이머 선택: https://github.com/arduino-libraries/Servo/blob/master/src/avr/ServoTimers.h

[7] Arduino 모터 제어 가이드: https://docs.arduino.cc/learn/electronics/transistor-motor-control/

[8] Arduino 서보 가이드: https://docs.arduino.cc/learn/electronics/servo-motors/

[9] ArduinoCore-avr Tone.cpp (Uno Timer2): https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/Tone.cpp

현재 로컬 프로젝트가 입증한 것

로컬 자료	현재 증거	새 자동차 제어에서의 의미
README.md / VERIFICATION.md	1보드 문자 생성 컴파일·호스트/mock 시험	빌드·검증 흐름 참고, 제어 성능 아님
model_metadata.json / model.npz	문맥16·H48·29문자 다음 문자 모델	새 단어·10의도 모델과 구조·목표가 다름
tiny_uno/ / tools/	기존 문자 모델의 PROGMEM·고정 버퍼 기반	새 제어·driver·4보드 구현은 별도
intent/	V3 학습·export·C++ 수치 비교·Uno probe 빌드	실제4보드·GPIO/tone·실물은 아직 미구현
docs/project-plan.md	이번 설계·규격·시험 기준	모델 호스트 검증과 실물·동시통신·ACK 검증을 구분

Arduino는 미연결·미업로드이며4보드 통신·모터·LED·서보·피에조 실물은 미검증이다. 호스트 모델·probe 빌드 완료와 자동차 완성을 구분한다. 피에조 사용만 정했고 수동/능동·모델명 등 부품은 미정이다.4보드 clock·타이밍은 설계 후보이다.

독립 감사가 보여 준 초기 모델의 제한

실험	TRAIN 중복 제외 최초 감사	관찰·후속 용도
V1 ID54d239ec, E12/H48	74문장 정확도59.46%	OOV·거절·FORWARD→WIPER_ON1건. 이후 회귀시험.
V2 IDf044b79e, E16/H64	65문장 정확도80.00%, 정상45개71.11%	OOV 중심, 위험 동작 수용0건. 이후 회귀시험.
V3 ID30918170, E16/H64	새49문장 전체 pipeline49/49 정확	TRAIN 중복2개 제외. 어휘 내raw39/42=92.86%; guard·전처리 포함.

V1/V2의 최초 감사와 V3 최초 감사는 서로 다른 문장 집합이다. V3의51개는 모델 공개 전 봉인하고 TRAIN 중복2개를 제외했다. 이전 감사의 V3 재평가는 회귀시험이며 OOV4개/2개가 남았다.

로컬 근거: intent/INDEPENDENT_VERIFICATION.md · intent/verification/final_v3_summary.json.